CSIT6910

Independent Project Report

R2T-Based Differentially Private Query Evaluation on TPC-H

Name: Xu Dongliu
Student ID: 21208710

Instructor: Ke Yi
Semester: 2025-26 Spring

May 26, 2026

Contents

1 Project Overview **2**
- 1.1 Project Title . . . 2
- 1.2 Background . . . 2
- 1.3 Project Scope and Objectives . . . 2
- 1.4 TPC-H Cases Used in the Project . . . 3

2 Algorithm Design **4**
- 2.1 Algorithm Overview . . . 4
- 2.2 Algorithm Detail . . . 5
- 2.3 Implementation Details . . . 5
- 2.4 Q3-Style SQL Query . . . 6
- 2.5 Q5-Style SQL Query . . . 6
- 2.6 Implementation Structure . . . 7

3 Experiment Design and Outcome **8**
- 3.1 Experiment Overview . . . 8
- 3.2 Input Design . . . 11
- 3.3 Answer Check Design . . . 12
- 3.4 Running Commands . . . 12
- 3.5 Experiment Results . . . 13

4 Conclusion **14**
- 4.1 Conclusion . . . 14

A Sample of Minutes **15**
- A.1 Minutes of the 1st Project Meeting . . . 15
- A.2 Minutes of the 2nd Project Meeting . . . 16
- A.3 Minutes of the 3rd Project Meeting . . . 17
- A.4 Minutes of the 4th Project Meeting . . . 18
- A.5 Project code . . . 19

Chapter 1

Project Overview

1.1 Project Title

R2T-Based Differentially Private Query Evaluation on TPC-H

1.2 Background

This project started from a practical question: if the private unit is a customer, how much can one customer change a join query on TPC-H? For a single table this kind of sensitivity bound is usually direct. For TPC-H it is less direct, because a customer can own many orders, and each order can have several lineitems. Removing one customer therefore also removes the tuples that reference that customer.

The R2T paper, *R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys*, addresses this problem by truncating private-unit contribution. A small threshold τ gives smaller noise but loses more of the query answer. A large τ keeps more revenue but also needs larger noise. R2T tries powers of two for τ, adds noise and a penalty to each candidate, and then returns the winner of this private race.

I did not try to build a full private SQL engine. That would require a much larger parser and query-rewriting system. Instead, I selected two TPC-H revenue queries where the private unit can be handled cleanly at the customer level. The code saves the intermediate candidate values, so the result can be checked instead of only reported as a final number.

1.3 Project Scope and Objectives

The final scope was kept narrow on purpose:

1. Use `Customer` as the private relation in both cases.
2. Implement two concrete TPC-H-style revenue queries rather than a general query framework.
3. Apply R2T to per-customer revenue contributions.
4. Save enough intermediate output to make the experiment reproducible.

5. Add a small validation script for the properties that are easy to check from the generated candidate files.

The protected unit is one customer together with the orders and lineitems that reference that customer. The two implemented queries follow the join and filter ideas of TPC-H Query 3 and Query 5, but they output one scalar revenue value instead of the original benchmark outputs. This keeps the project focused on the R2T truncation mechanism.

1.4 TPC-H Cases Used in the Project

The first case is a Q3-style customer revenue query. It joins `Customer`, `Orders`, and `Lineitem`. The query keeps customers in the `BUILDING` market segment, orders before 1995-03-15, and lineitems shipped after 1995-03-15. The revenue term is `L_EXTENDEDPRICE * (1 - L_DISCOUNT)`.

The second case is a Q5-style ASIA revenue query. It joins `Customer`, `Orders`, `Lineitem`, `Supplier`, `Nation`, and `Region`. It keeps orders in 1994, restricts the region to ASIA, and requires the supplier and customer to come from the same nation.

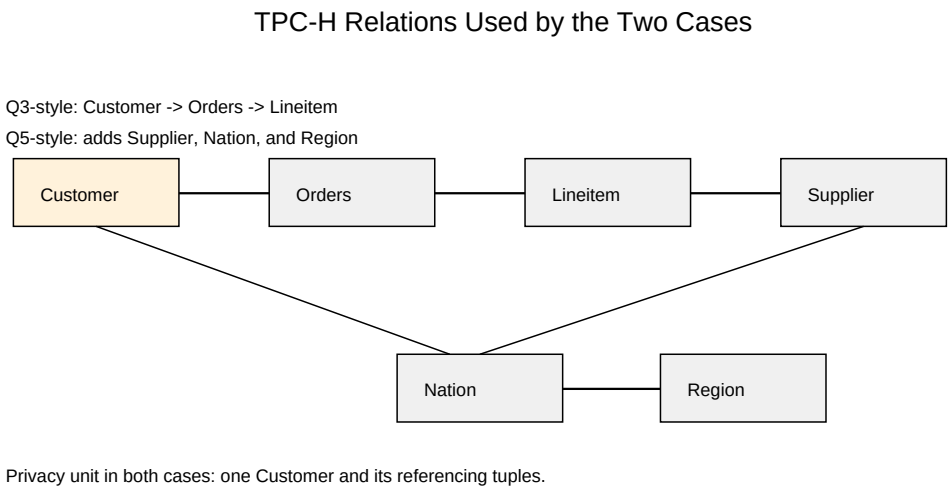

Figure 1.1: Foreign key relationship and TPC-H relations used in the project

The local TPC-H data files are stored under `data/`. The scripts use that directory by default, and `--data-dir` can be used when a different generated TPC-H dataset is needed.

Chapter 2

Algorithm Design

2.1 Algorithm Overview

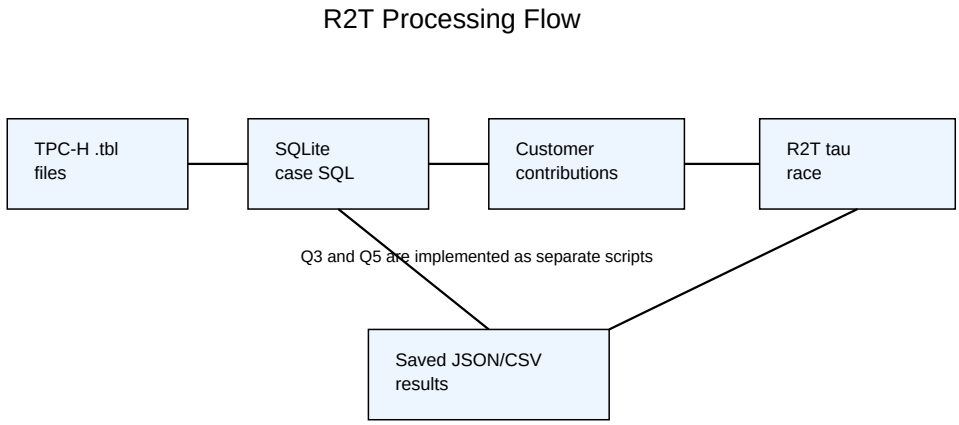

Figure 2.1: Algorithm flow

The pipeline in Figure 2.1 is the one used in the experiment. Each case loads only the tables it needs into an in-memory SQLite database. The SQL query then returns one row per contributing customer. After that point, the database part is finished; R2T only sees a list of customer revenue values.

Let x_i denote the contribution of customer i to the selected query. For a threshold τ, the truncated query answer is

$$Q(I, \tau) = \sum_i \min(x_i, \tau).$$

This is the main simplification in the project. The two selected cases are self-join-free, and every released value is first reduced to a customer contribution. Under this setup, capping each customer's revenue at τ gives the truncation function needed by R2T. The

full paper uses LP formulations for more complicated settings, but those extra pieces are not needed for these two selected queries.

2.2 Algorithm Detail

The implementation follows these steps:

1. **Source Loading**: Read selected TPC-H `.tbl` files with Python's `csv` module and insert them into SQLite.
2. **Case Query Execution**: Run one fixed SQL query per case. The output is grouped by customer key.
3. **Contribution Construction**: Convert the SQL rows into `(customer_key, contribution)` pairs. Their sum is the non-private SQL answer.
4. **Threshold Enumeration**: Generate candidate thresholds $\tau = 2, 4, 8, \ldots, 2^{\lfloor \log_2(GS_Q) \rfloor}$. The default value is $GS_Q = 10{,}000{,}000$.
5. **Truncated Answer**: For each threshold, compute $\sum_i \min(x_i, \tau)$. This value is monotone in τ and never exceeds the original SQL total.
6. **Noisy R2T Score**: Add Laplace noise and subtract the R2T penalty term. The score for threshold τ is

$$\widetilde{Q}(I,\tau) = Q(I,\tau) + Lap\left(\frac{\log_2(GS_Q)\tau}{\epsilon}\right) - \log_2(GS_Q)\ln\left(\frac{\log_2(GS_Q)}{\beta}\right)\frac{\tau}{\epsilon}.$$

7. **Final Output**: Return

$$\max\left(0, \max_{\tau} \widetilde{Q}(I,\tau)\right),$$

   where $Q(I,0) = 0$ because the revenue values are non-negative.
8. **Result Saving**: Save the summary, all candidate thresholds, and the customer contribution list under `results/`.

2.3 Implementation Details

The code is split into a shared module and two case scripts. `code/common.py` contains the loader, the R2T loop, printing, and file output. The loader reads the TPC-H `.tbl` files directly instead of using a dataframe library. TPC-H rows end with a trailing `|`, so the loader stores `row[:-1]` in SQLite. I used explicit column names when creating each table, because it makes the SQL queries easier to read.

After the tables are loaded, each case script creates SQLite indexes on the main join keys. For the Q3-style case, the indexes are on `Orders(O_CUSTKEY)` and `Lineitem(L_ORDERKEY)`. For the Q5-style case, extra indexes are created on `Lineitem(L_SUPPKEY)` and `Supplier(S_NATIONKEY)`. These indexes do not change the query result; they only make the full-data experiment faster.

The function `customer_contributions()` is the boundary between the SQL part and the R2T part. It returns `(customer_key, revenue)` pairs. Since the private unit is a customer, this is also the right level at which to apply truncation.

The function `r2t_from_contributions()` implements the actual R2T loop. It first computes `log2(GS_Q)`, then enumerates powers of two by calling `tau_candidates()`. For every threshold, it computes the truncated value with:

```
truncated = sum(min(value, tau) for _, value in contributions)
```

Then it samples Laplace noise using inverse transform sampling. A uniform random value is transformed into a Laplace random variable, so no external `numpy` dependency is required. The noisy R2T score is then computed as:

```
noise_scale = log_g * tau / epsilon
noisy_score = truncated + laplace(rng, noise_scale) -
   penalty_factor * tau
```

The best candidate is the row with the largest `noisy_score`. The final output applies the $Q(I, 0)$ lower bound:

```
r2t_output = max(best["noisy_score"], 0.0)
```

Finally, `save_result()` writes three output files. The JSON summary stores the selected threshold, final R2T output, true SQL total, maximum customer contribution, and parameters. One CSV file stores all candidate thresholds, including the truncated answer, noise scale, and noisy score. The other CSV file stores the per-customer contributions. This was useful for checking the result later and for drawing the contribution concentration figure.

2.4 Q3-Style SQL Query

```
SELECT
    c.C_CUSTKEY,
    SUM(l.L_EXTENDEDPRICE * (1.0 - l.L_DISCOUNT)) AS revenue
FROM Customer c
JOIN Orders o
    ON c.C_CUSTKEY = o.O_CUSTKEY
JOIN Lineitem l
    ON o.O_ORDERKEY = l.L_ORDERKEY
WHERE c.C_MKTSEGMENT = 'BUILDING'
  AND o.O_ORDERDATE < '1995-03-15'
  AND l.L_SHIPDATE > '1995-03-15'
GROUP BY c.C_CUSTKEY
HAVING revenue > 0;
```

This query follows the filtering idea of TPC-H Query 3, but I aggregate by customer instead of reporting individual orders. That change is needed because the privacy unit in this project is the customer.

2.5 Q5-Style SQL Query

```sql
SELECT
    c.C_CUSTKEY,
    SUM(l.L_EXTENDEDPRICE * (1.0 - l.L_DISCOUNT)) AS revenue
FROM Customer c
JOIN Orders o
    ON c.C_CUSTKEY = o.O_CUSTKEY
JOIN Lineitem l
    ON o.O_ORDERKEY = l.L_ORDERKEY
JOIN Supplier s
    ON l.L_SUPPKEY = s.S_SUPPKEY
JOIN Nation n
    ON c.C_NATIONKEY = n.N_NATIONKEY
   AND s.S_NATIONKEY = n.N_NATIONKEY
JOIN Region r
    ON n.N_REGIONKEY = r.R_REGIONKEY
WHERE r.R_NAME = 'ASIA'
  AND o.O_ORDERDATE >= '1994-01-01'
  AND o.O_ORDERDATE < '1995-01-01'
GROUP BY c.C_CUSTKEY
HAVING revenue > 0;
```

This query follows the ASIA-region and same-nation condition from TPC-H Query 5. The original query groups by nation. Here the grouping is by customer, again so that the contribution of one private customer can be capped by R2T.

2.6 Implementation Structure

The implementation is intentionally direct. There is no query parser and no general SQL rewriting module. The files are:

- `code/common.py`: shared table loading, threshold generation, Laplace sampling, R2T scoring, printing, and result saving.
- `code/case_q3_customer_revenue.py`: the Q3-style case.
- `code/case_q5_asia_revenue.py`: the Q5-style case.
- `code/validate_cases.py`: lightweight validation checks.
- `code/generate_report_figures.py`: figure generation from saved CSV/JSON results and saved contribution files.

The runtime path uses only the Python standard library and SQLite. I kept it this way so the experiment can run in a plain Python environment without setting up extra numerical packages.

Chapter 3

Experiment Design and Outcome

3.1 Experiment Overview

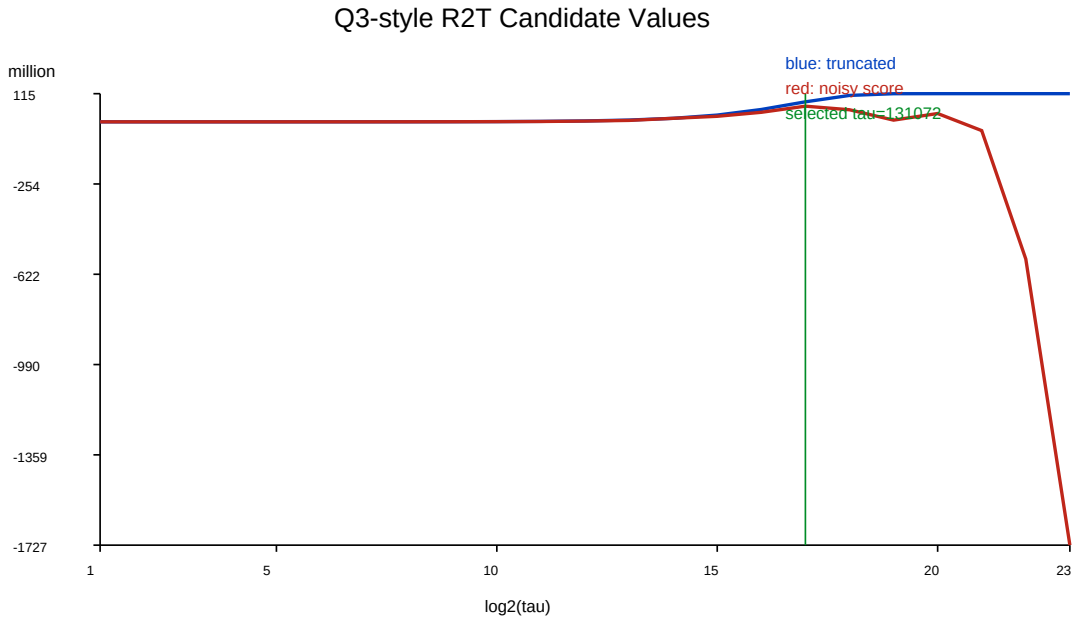

Figure 3.1: Experiment outcome for the Q3-style case

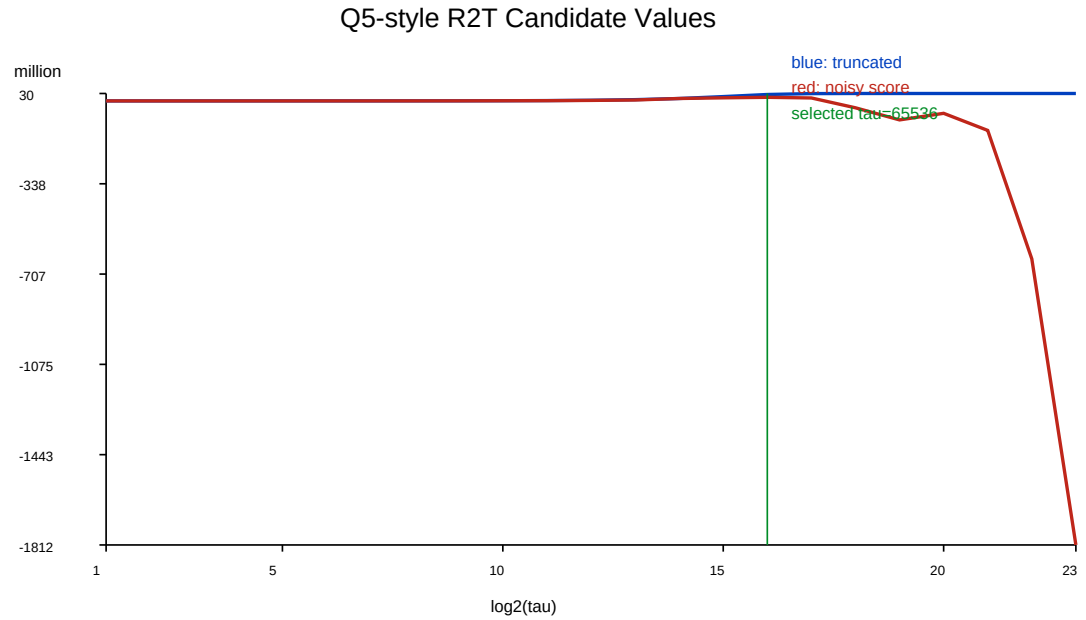

Figure 3.2: Experiment outcome for the Q5-style case

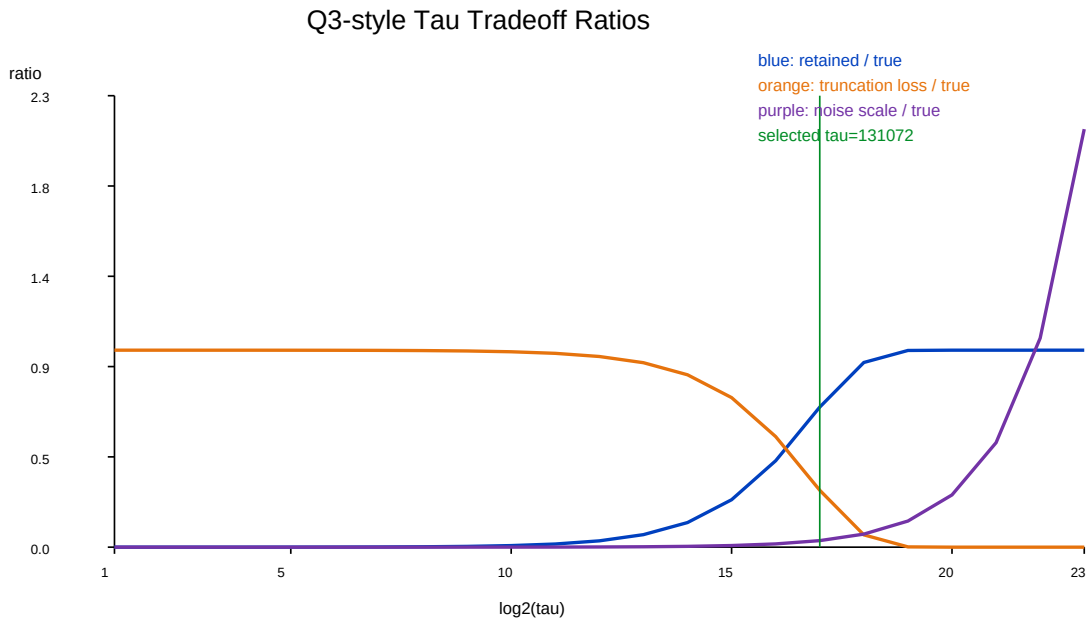

Figure 3.3: Q3-style threshold tradeoff between retained answer, truncation loss, and noise scale

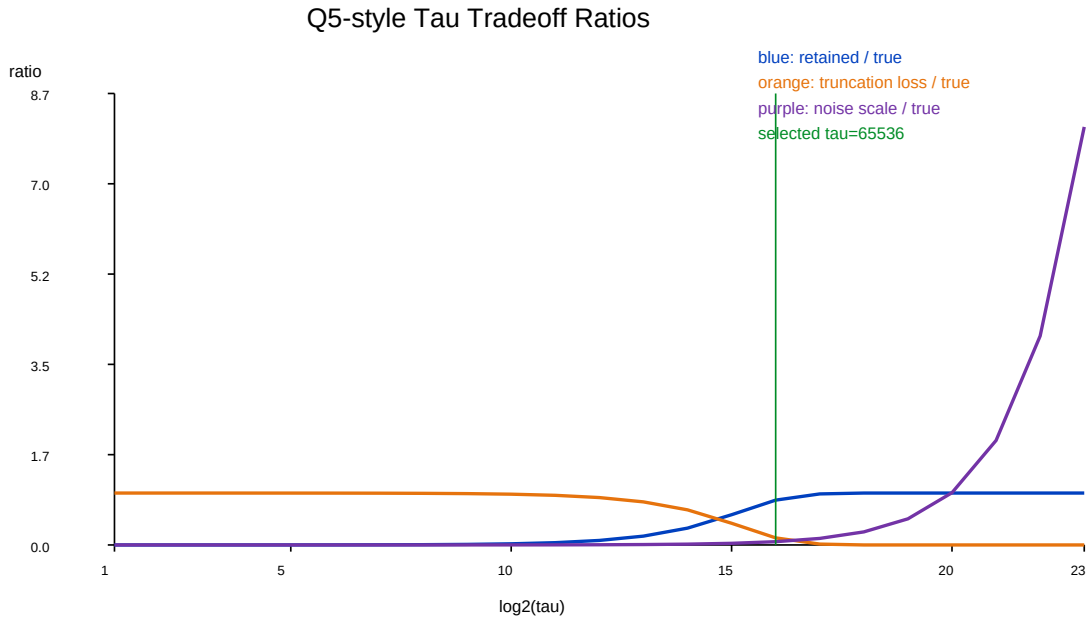

Figure 3.4: Q5-style threshold tradeoff between retained answer, truncation loss, and noise scale

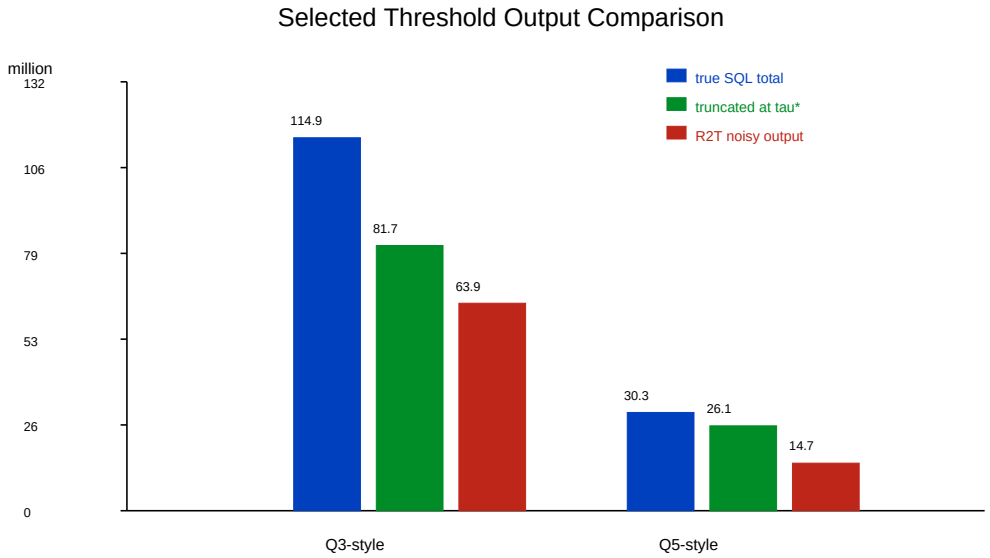

Figure 3.5: Comparison of true SQL total, selected-threshold truncated value, and final R2T noisy output

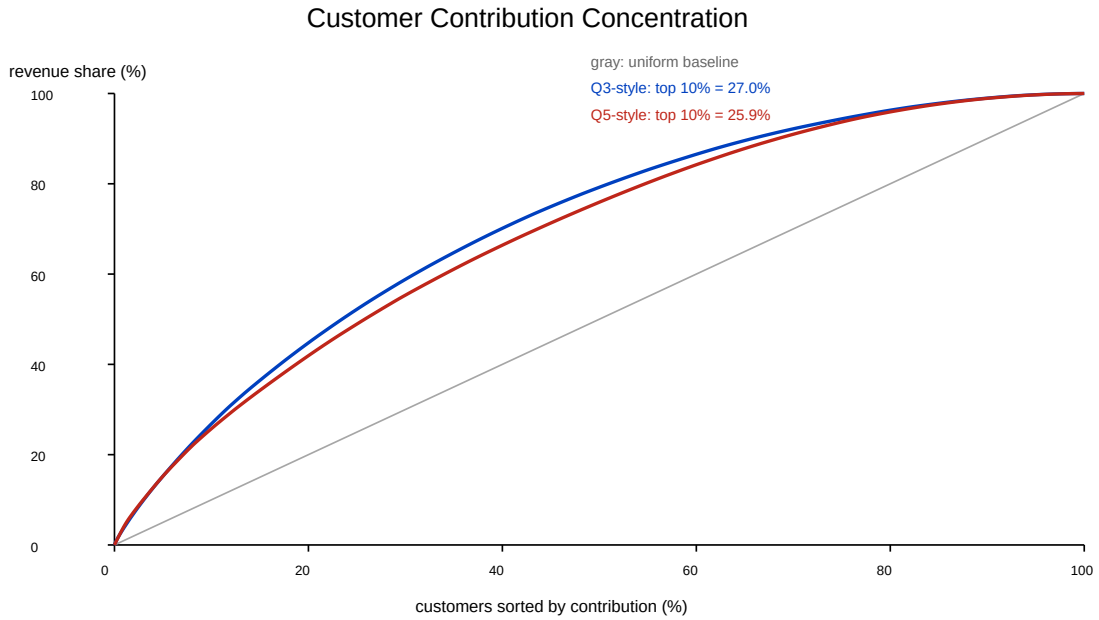

Figure 3.6: Customer contribution concentration for the two implemented cases

The experiment runs both cases on the local TPC-H scale-1 data. For each case I record the true SQL total before privacy, the maximum customer contribution, the selected threshold τ, the truncated value at that threshold, and the final noisy R2T output. The candidate CSV files make the threshold race visible, while the contribution CSV files show how concentrated the customer revenues are.

3.2 Input Design

1. Use the TPC-H `.tbl` files stored in `data/`. The tables used are `customer.tbl`, `orders.tbl`, `lineitem.tbl`, `supplier.tbl`, `nation.tbl`, and `region.tbl`.

2. Load only the tables required by each case. The Q3-style case loads `Customer`, `Orders`, and `Lineitem`. The Q5-style case also loads `Supplier`, `Nation`, and `Region`.

3. Store tables in SQLite memory tables with explicit column names based on the TPC-H schema. This keeps the SQL readable and avoids an external database service.

4. Create indexes on the main join keys, such as `O_CUSTKEY`, `L_ORDERKEY`, and `L_SUPPKEY`. These indexes reduce the runtime of the full data query.

5. Use fixed R2T parameters for reproducibility: $\epsilon = 0.8$, $\beta = 0.1$, $GS_Q = 10{,}000{,}000$, and random seed 1.

3.3 Answer Check Design

The project includes a validation script:

```
python3 code/validate_cases.py
```

The validation script is not a formal DP proof. Its purpose is more practical: rerun the two cases and catch mistakes in the truncation values or saved candidate rows. It checks:

1. Each case produces at least one contributing customer.
2. The true SQL total is positive.
3. The truncated value is monotone non-decreasing as τ increases.
4. The truncated value never exceeds the original SQL total.
5. The truncated value is bounded by $\tau\times$ the number of contributing customers.

These checks passed for both implemented cases. This is important because the R2T mechanism relies on the truncated value being a stable underestimate of the true answer.

More specifically, the validation script reruns both cases on a smaller default sample and checks these invariants inside `validate_result()`:

```
assert result["private_entities"] > 0
assert result["true_total"] > 0
assert result["best"]["tau"] > 0

previous = -1.0
for row in result["rows"]:
    truncated = row["truncated"]
    tau = row["tau"]
    assert truncated >= previous
    assert truncated <= result["true_total"] + 1e-6
    assert truncated <= tau * result["private_entities"] + 1e-6
    previous = truncated
```

The first three assertions check that the SQL query produced a usable case and that R2T selected a positive threshold. The monotonicity check matches the formula $\sum_i \min(x_i, \tau)$: increasing τ cannot decrease the truncated answer. The upper-bound check confirms that truncation never creates extra revenue. The last bound checks that the total capped contribution cannot exceed $\tau\times$ the number of contributing customers. When the script was run, both cases returned `ok`.

3.4 Running Commands

The main commands used in the experiment are:

```
python3 code/case_q3_customer_revenue.py --seed 1
python3 code/case_q5_asia_revenue.py --seed 1
python3 code/validate_cases.py
python3 code/generate_report_figures.py
```

Each case writes three result files. The summary file records the selected threshold and final output, the candidate file records every threshold evaluated by R2T, and the contribution file records each customer's pre-truncation revenue.

3.5 Experiment Results

Metric	**Q3-style**	**Q5-style**
Contributing customers	904	655
True SQL total	114,904,912.53	30,276,617.68
Maximum customer contribution	661,967.22	210,018.26
Selected τ	131,072	65,536
Truncated value at selected τ	81,707,181.04	26,148,964.08
Noise scale at selected τ	3,809,852.89	1,904,926.45
R2T noisy output	63,926,157.02	14,653,924.70

Table 3.1: Full-data experimental results with seed 1

For the Q3-style case, the true SQL total is about 114.90 million. The largest customer contributes about 0.66 million. R2T selects $\tau = 131{,}072$, so the selected truncated answer is about 81.71 million before noise and penalty. A larger threshold would recover more of the SQL answer, but it would also increase the penalty and the noise scale. This is why the selected threshold is below the value needed to recover the full total.

For the Q5-style case, the true SQL total is about 30.28 million and the largest customer contribution is about 0.21 million. The selected threshold is $\tau = 65{,}536$, with a truncated value of about 26.15 million. Compared with Q3, this case has fewer contributing customers and a smaller revenue scale, so the selected threshold and noise scale are also lower.

Chapter 4

Conclusion

4.1 Conclusion

The project implements R2T for two concrete TPC-H revenue queries with customer-level privacy. The important part is the reduction from a join query to per-customer revenue contributions. After that reduction, the R2T threshold race can be implemented directly with $\sum_i \min(x_i, \tau)$.

The experiment also shows the expected trade-off. Larger thresholds keep more of the true answer, but they increase both the penalty term and the Laplace noise scale. In both cases, R2T chooses an intermediate threshold instead of the largest possible threshold.

The main limitation is that this is not a general private SQL engine. It does not handle arbitrary SQL, self-joins, or projection-heavy SPJA queries. Within the two selected cases, however, the implementation gives a reproducible path from TPC-H tables to saved R2T outputs and figures.

Appendix A

Sample of Minutes

A.1 Minutes of the 1st Project Meeting

Meeting Minutes

Date: Saturday, February 7, 2026
Time: 4:00 pm
Place: Zoom meeting
Present: Xu Dongliu
Apology: None
Note-taker: Xu Dongliu

1. Approval of Minutes

This was the first project meeting, so there were no minutes to approve.

2. Discussion Items

Background Of Project

We discussed the R2T paper and how to use TPC-H data for a differential privacy project. The initial plan was to understand the foreign-key privacy model and decide which TPC-H queries could be implemented within the project scope.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 4:45 pm. The next meeting was planned for March.

A.2 Minutes of the 2nd Project Meeting

Meeting Minutes

Date:	Tuesday, March 17, 2026
Time:	3:00 pm
Place:	Zoom meeting
Present:	Xu Dongliu
Apology:	None
Note-taker:	Xu Dongliu

1. Approval of Minutes

The minutes of the first meeting were reviewed. No changes were needed.

2. Discussion Items

Background Of Project

We discussed the project progress. The TPC-H schema and R2T truncation mechanism had been reviewed. The project direction was adjusted to implement two concrete TPC-H cases rather than a general query framework. The selected privacy relation was Customer.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 3:45 pm. The next meeting was planned for April.

A.3 Minutes of the 3rd Project Meeting

Meeting Minutes

Date:	Tuesday, April 21, 2026
Time:	4:30 pm
Place:	Zoom meeting
Present:	Xu Dongliu
Apology:	None
Note-taker:	Xu Dongliu

1. Approval of Minutes

The minutes of the second meeting were reviewed. No changes were needed.

2. Discussion Items

Background Of Project

We discussed the implementation progress. The Q3-style and Q5-style SQL queries had been written, and the code could load TPC-H tables into SQLite. The next step was to finish the R2T threshold loop, save outputs, and add validation checks.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 5:00 pm. The next meeting was planned for May.

A.4 Minutes of the 4th Project Meeting

Meeting Minutes

Date:	Tuesday, May 19, 2026
Time:	3:00 pm
Place:	Zoom meeting
Present:	Xu Dongliu
Apology:	None
Note-taker:	Xu Dongliu

1. Approval of Minutes

The minutes of the third meeting were reviewed. No changes were needed.

2. Discussion Items

Background Of Project

We discussed the experiment outcome and final report. The two R2T cases had been executed on the full TPC-H data. The validation script passed, and figures were generated for submission.

3. Meeting Adjournment and Next Meeting

The meeting adjourned at 5:00 pm. This was the last meeting.

A.5 Project code

https://github.com/xdl2003/r2t-tpch-dp